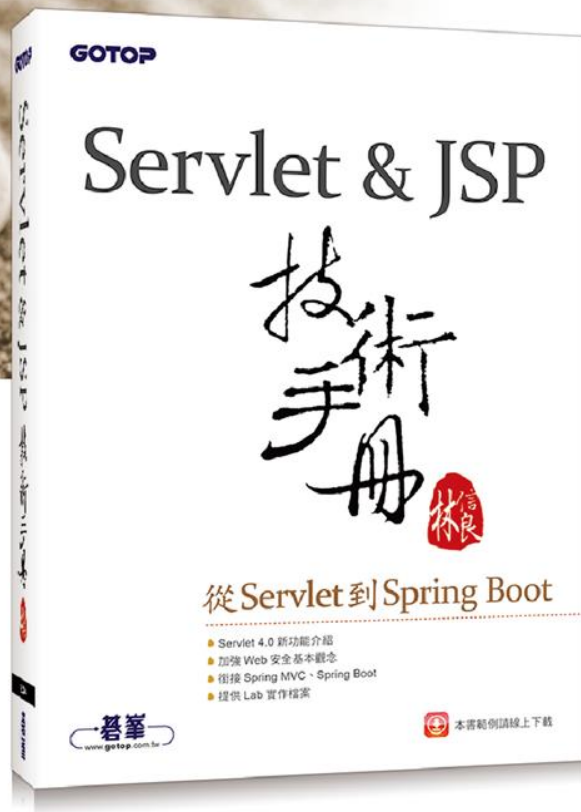




Servlet & JSP

技術手冊

從Servlet到Spring Boot

- Servlet 4.0 新功能介紹
- 加強 Web 安全基本觀念
- 銜接 Spring MVC、Spring Boot
- 提供 Lab 實作檔案

• 整合資料庫

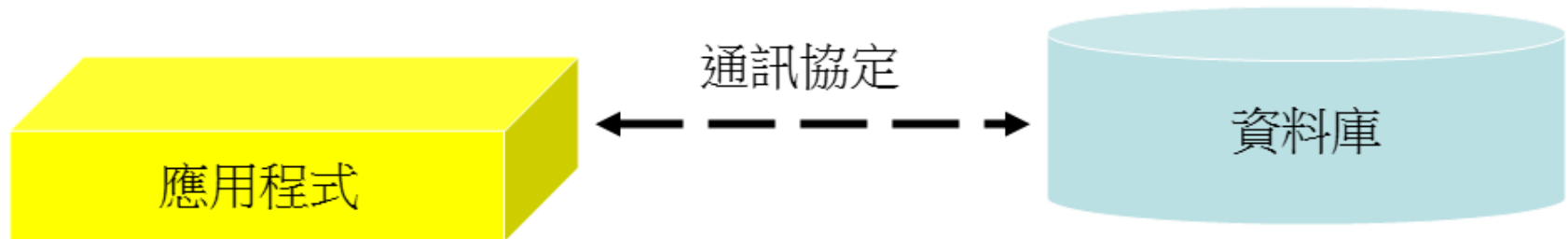


學習目標

- 了解JDBC架構
- 使用基本的JDBC
- 透過JNDI取得DataSource
- 在Web應用程式整合資料庫

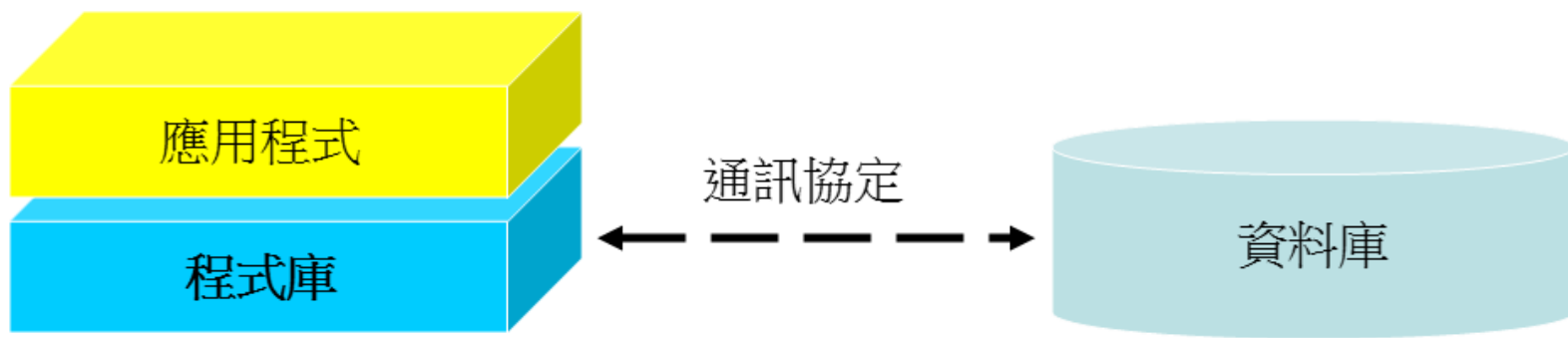
簡介JDBC

- 資料庫本身是個獨立運行的應用程式
- 撰寫的應用程式是利用網路通訊協定與資料庫進行指令交換，以進行資料的增刪查找



簡介JDBC

- 應用程式會利用一組專門與資料庫進行通訊協定的程式庫



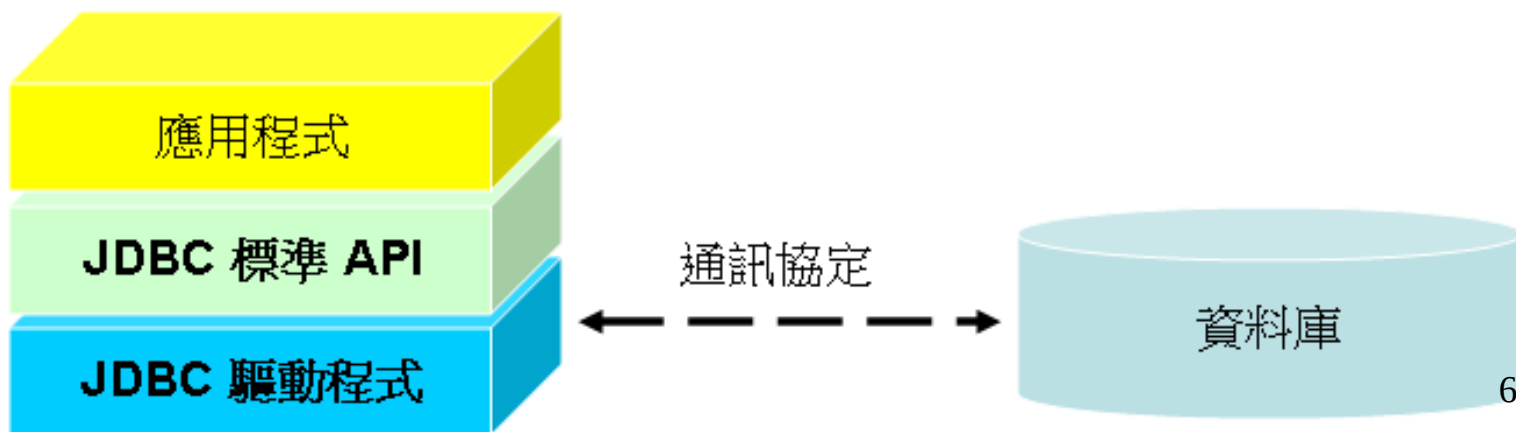
簡介JDBC

- 應用程式如何呼叫這組程式庫？
- 不同的資料庫通常會有不同的通訊協定
- 用以連線不同資料庫的程式庫在API上也會有所不同

```
XySqlConnection conn = new XySqlConnection("localhost", "root", "1234");  
conn.selectDB("gossip");  
XySqlQuery query = conn.query("SELECT * FROM T_USER");
```

簡介JDBC

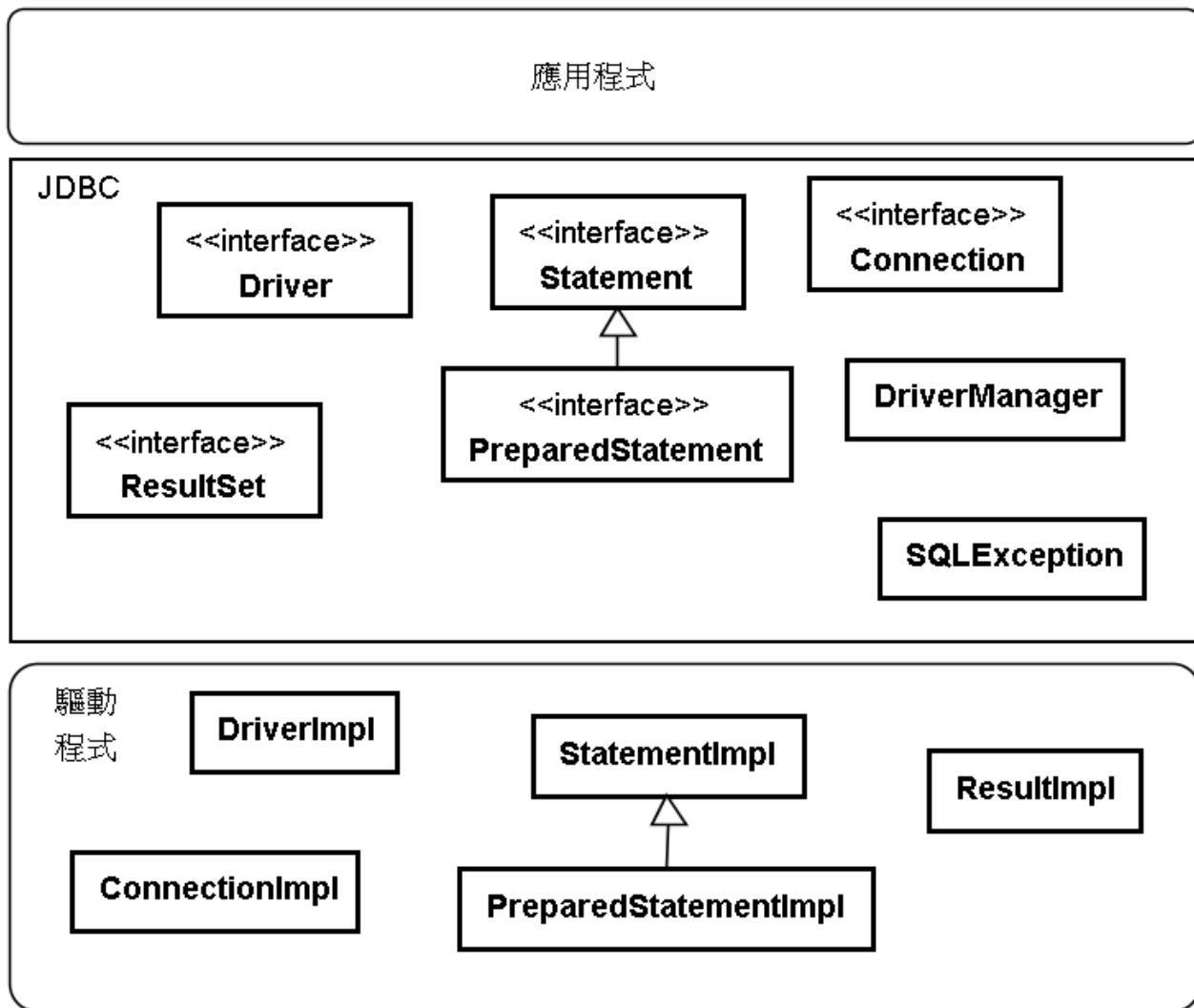
- JDBC全名Java DataBase Connectivity
- Java連線資料庫的標準規範，定義一組標準類別與介面：
 - 應用程式需要連線資料庫時就呼叫這組標準API，而標準API中的介面會由資料庫廠商實作，通常稱之為JDBC驅動程式（Driver）



簡介JDBC

- JDBC應用程式開發者介面（Application Developer Interface）
 - 應用程式需要連線資料庫
 - 相關API主要是座落於`java.sql`與`javax.sql`
- JDBC驅動程式開發者介面（Driver Developer Interface）
 - 資料庫廠商要實作驅動程式時的規範，一般開發者並不用瞭解

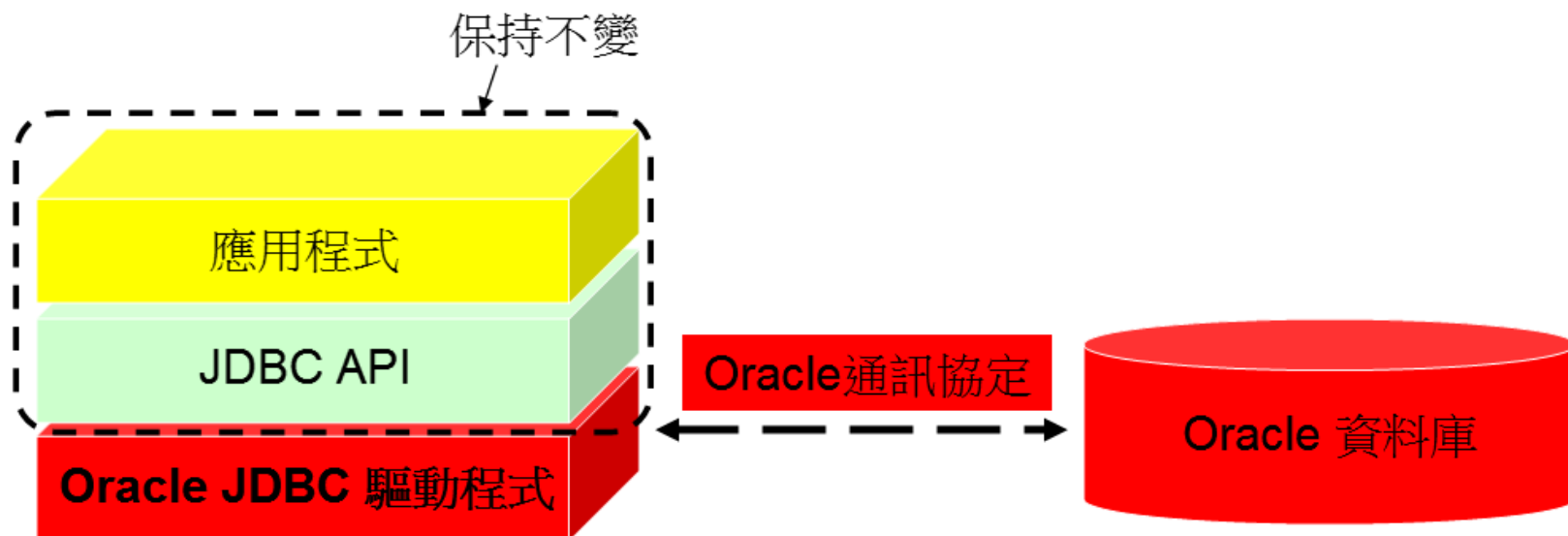
簡介JDBC



簡介JDBC

- 應用程式會使用JDBC連線資料庫

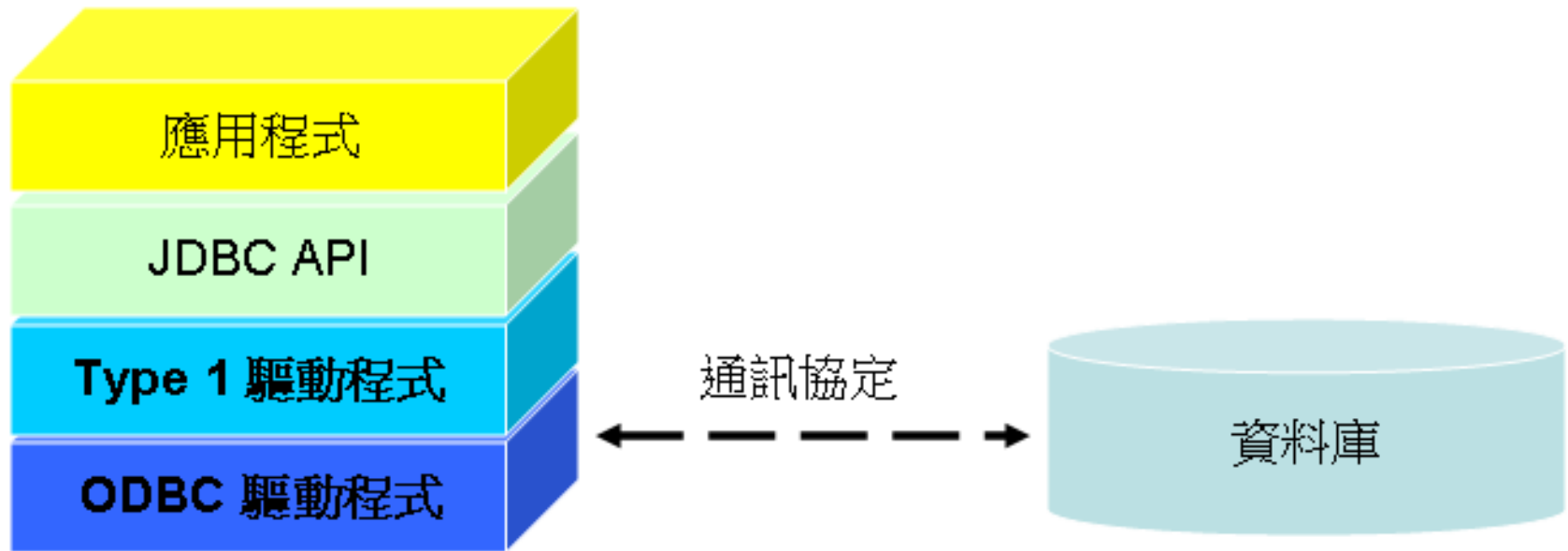
```
Connection conn = DriverManager.getConnection(...);  
Statement st = conn.createStatement();  
ResultSet rs = st.executeQuery("SELECT * FROM T_USER");
```



簡介JDBC

- 廠商在實作JDBC驅動程式時，依方式可將驅動程式分作四種類型
 - Type 1：JDBC-ODBC Bridge Driver
 - Type 2：Native API Dirver
 - Type 3：JDBC-Net Driver
 - Type 4：Native Protocol Driver

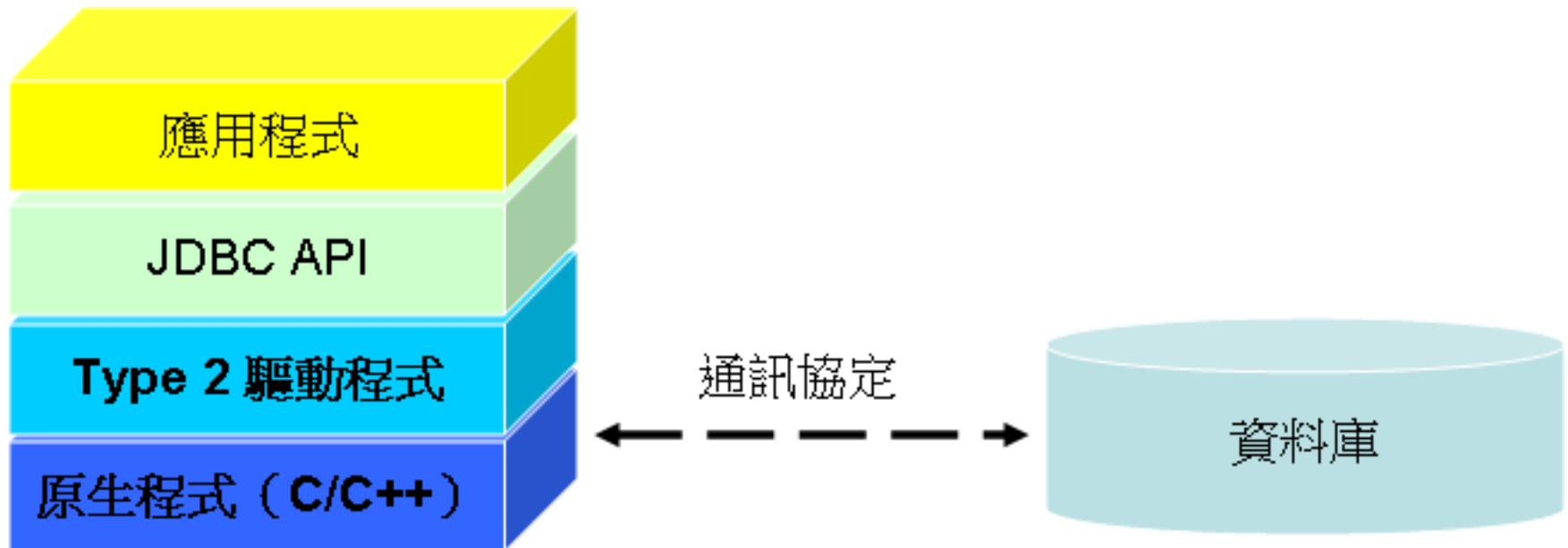
Type 1 : JDBC-ODBC Bridge Driver



Type 1 : JDBC-ODBC Bridge Driver

- 實作這種驅動程式非常簡單
- JDBC與ODBC並非一對一的對應，所以部份呼叫無法直接轉換，因此有些功能是受限的
- 多層呼叫轉換結果，存取速度也會受到限制
- ODBC本身需在平台上先設定好，彈性不足，ODBC驅動程式本身也有跨平台的限制

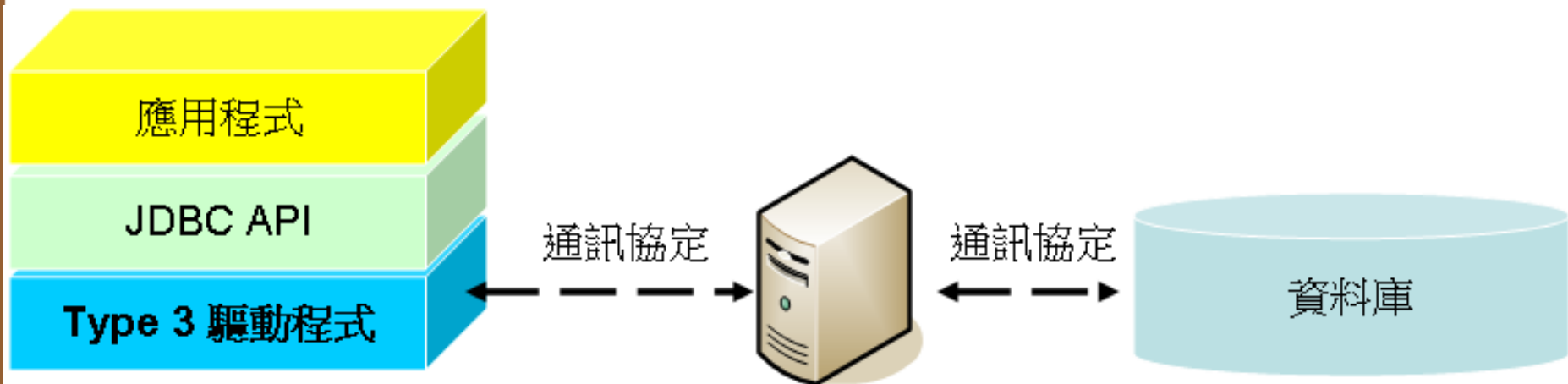
Type 2 : Native API Dirver



Type 2 : Native API Dirver

- 驅動程式本身與平台相依，沒有達到JDBC驅動程式的目標之一：跨平台
- 直接呼叫資料庫原生API，因此在速度上，有機會成為四種類型中最快的驅動程式
 - 速度的優勢是在於獲得資料庫回應資料後，建構相關 JDBC API 實作物件時
- 使用前必須先在各平台進行驅動程式的安裝設定（像是安裝資料庫專屬的原生程式庫）

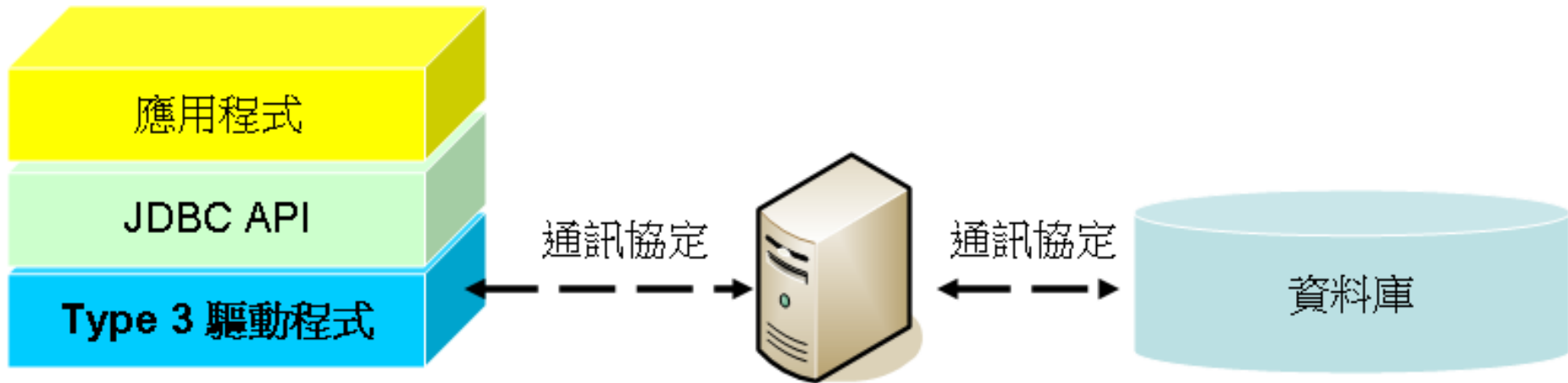
Type 3 : JDBC-Net Driver



Type 3 : JDBC-Net Driver

- 可使用純粹的Java技術來實現，可以跨平台
- 架構彈性高，客戶端不受影響
- 經由中介伺服器轉換，速度較慢，獲得架構上的彈性是使用這類型驅動程式的目的

Type 4 : Native Protocol Driver



Type 4 : Native Protocol Driver

- 驅動程式可以使用純粹Java技術來實現，可以跨平台
- 效能上也能有不錯的表現
- 不需要如Type 3獲得架構上的彈性時，通常會使用這類型驅動程式

連接資料庫

- 基本資料庫操作相關的JDBC介面或類別是位於`java.sql`套件中
- 要取得資料庫連線，必須有幾個動作：
 - 註冊`Driver`實作物件
 - 取得`Connection`實作物件
 - 關閉`Connection`實作物件

註冊Driver實作物件

- 以MySQL實作的驅動程式為例，
`org.h2.Driver`類別實作了
`java.sql.Driver`介面
- 管理Driver實作物件的類別是
`java.sql.DriverManager`

```
DriverManager.registerDriver(new org.h2.Driver());
```

註冊Driver實作物件

- 實際上很少自行撰寫程式碼進行這個動作
- 只要想辦法載入Driver介面的實作類別.class檔案，就會完成註冊
- 可以透過`java.lang.Class`類別的`forName()`，動態載入驅動程式類別

```
try {  
    Class.forName("org.h2.Driver");  
}  
catch(ClassNotFoundException e) {  
    throw new RuntimeException("找不到指定的類別");  
}
```

註冊Driver實作物件

```
package org.h2;
...略

public class Driver implements java.sql.Driver, JdbcDriverBackwardsCompat {
    private static final Driver INSTANCE = new Driver();
    ...略

    static {
        load();
    }

    public static synchronized Driver load() {
        try {
            if (!registered) {
                registered = true;
                DriverManager.registerDriver(INSTANCE);
            }
        } catch (SQLException e) {
            DbException.traceThrowable(e);
        }
        return INSTANCE;
    }
}
```

註冊Driver實作物件

- 使用JDBC時，要求載入.class檔案的方式有四種：

- 使用Class.forName()
- 自行建立Driver介面實作類別的實例

```
java.sql.Driver driver = new org.h2.Driver();
```

- 啟動JVM時指定jdbc.drivers屬性

```
> java -Djdbc.drivers=org.h2.Driver;ooo.XXXDriver YourProgram
```

- 設定JAR中 /services/java.sql.Driver 檔案（JDK6）

取得Connection實作物件

```
Connection conn = DriverManager.getConnection(  
    jdbcUrl, username, password);
```

協定:子協定:資料來源識別



- 除了「協定」在JDBC URL中總是jdbc開始之外，JDBC URL格式各家資料庫都不相同，必須查詢資料庫產品使用手冊


```
Connection conn = null;
SQLException ex = null;
try {
    String uri = "jdbc:h2:tcp://localhost/c:/workspace/JDBC/demo";
    String user = "caterpillar";
    String password = "12345678";
    conn = DriverManager.getConnection(uri, user, password);
    ....
}
catch(SQLException e) {
    ex = e;
}
finally {
    if(conn != null) {
        try {
            conn.close();
        }
        catch(SQLException e) {
            if(ex == null) {
                ex = e;
            }
        }
    }
    if(ex != null) {
        throw new RuntimeException(ex);
    }
}
```

SQLException

- 在處理JDBC時很常遇到的一個例外物件，為資料庫操作過程發生錯誤時的代表物件
- 受檢例外（Checked Exception），必須使用 `try...catch` 明確處理，在例外發生時嘗試關閉相關資源

關閉Connection實作物件

- 取得Connection物件之後，可以使用 **isClosed()** 方法測試與資料庫的連接是否關閉
- 若確定不再需要連接，則必須使用 `close()` 來關閉與資料庫的連接

取得Connection實作物件

- DriverManager如何進行連線？

```
SQLException reason = null;
for (int i = 0; i < drivers.size(); i++) { // 逐一取得 Driver 實例
    ...
    DriverInfo di = (DriverInfo)drivers.elementAt(i);
    ...
    try {
        Connection result = di.driver.connect(url, info); // 嘗試連線
        if (result != null) {
            return (result); // 取得 Connection 就傳回
        }
    } catch (SQLException ex) {
        if (reason == null) { // 記錄第一個發生的例外
            reason = ex;
        }
    }
}
if (reason != null) {
    println("getConnection failed: " + reason);
    throw reason; // 如果有例外物件就丟出
}
throw new SQLException( // 沒有適用的 Driver 實例，丟出例外
    "No suitable driver found for "+ url, "08001");
```

使用Statement、ResultSet

- **java.sql.Statement**物件是SQL陳述的代表物件
- 可以使用Connection的
createStatement()來建立

```
Statement stmt = conn.createStatement();
```

- 可以使用**executeUpdate()**、
executeQuery()等方法來執行 SQL

使用Statement、ResultSet

- `executeUpdate()` 主要是用來執行
CREATE TABLE、INSERT、DROP TABLE、
ALTER TABLE等會改變資料庫內容的SQL

```
stmt.executeUpdate("INSERT INTO t_message VALUES(1, 'justin', " +  
    "'justin@mail.com', 'message...')");
```

- `executeQuery()` 方法則是用於SELECT等
查詢資料庫的SQL

使用Statement、ResultSet

- `executeUpdate()` 會傳回`int`結果，表示資料變動的筆數
- `executeQuery()` 會傳回`java.sql.ResultSet`物件，代表查詢的結果

```
ResultSet result = stmt.executeQuery("SELECT * FROM t_message");
while(result.next()) {
    int id = result.getInt("id");
    String name = result.getString("name");
    String email = result.getString("email");
    String msg = result.getString("msg");
    // ...
}
```


使用Statement、ResultSet

```
ResultSet result = stmt.executeQuery("SELECT * FROM t_message");
while(result.next()) {
    int id = result.getInt(1);
    String name = result.getString(2);
    String email = result.getString(3);
    String msg = result.getString(4);
    // ...
}
```

使用Statement、ResultSet

- Statement的**execute()**可以用來執行SQL
 - 傳回true的話表示將傳回查詢結果，可以使用**getResultSet()**取得ResultSet物件
 - 傳回false，表示傳回更新筆數或沒有結果，可以使用**getUpdateCount()**取得更新筆數。

```
if(stmt.execute(sql)) {  
    ResultSet rs = stmt.getResultSet(); // 取得查詢結果 ResultSet  
    ...  
}  
else { // 這是個更新操作  
    int updated = stmt.getUpdateCount(); // 取得更新筆數  
    ...  
}
```

```
public void setMessage(Message message) { ← ❶ 這個方法會在資料庫中新增留言
    try(Connection conn = DriverManager.getConnection( ← ❷ 取得 Connection
        jdbcUri, username, password);
        Statement statement = conn.createStatement()) { ← ❸ 建立 Statement

        statement.executeUpdate( ← ❹ 執行 SQL 陳述句
            "INSERT INTO t_message(name, email, msg) VALUES ('"
                + message.getName() + "', '"
                + message.getEmail() + "', '"
                + message.getMsg() + "')");
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

```
public List<Message> getMessages() { ← ❸ 這個方法會從資料庫中查詢所有留言
    try(Connection conn = DriverManager.getConnection(
        jdbcUri, username, password);
        Statement statement = conn.createStatement()) {
        ResultSet result = statement.executeQuery("SELECT * FROM t_message");
        List<Message> messages = new ArrayList<>();
        while (result.next()) {
            Message message = new Message();
            message.setId(result.getLong(1));
            message.setName(result.getString(2));
            message.setEmail(result.getString(3));
            message.setMsg(result.getString(4));
            messages.add(message);
        }
        return messages;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
```

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<c:set target="${pageContext.request}"
    property="characterEncoding" value="UTF-8"/> ← 設定請求編碼處理
<jsp:useBean id="guestbook" ← 使用 GuestBookBean      方式為 UTF-8
    class="cc.openhome.GuestBookBean" scope="application"/>
<c:if test="${param.msg != null}"> ← 如果是要新增留言的話
    <jsp:useBean id="newMessage" class="cc.openhome.Message"/>
    <jsp:setProperty name="newMessage" property="*" />
    <c:set target="${guestbook}" ← 呼叫 setMessage() 方法新增留言
        property="message" value="${newMessage}" />
</c:if>
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>訪客留言版</title>
    </head>
    <body>
        <table style="text-align: left; width: 100%;" border="0"
            cellpadding="2" cellspacing="2">
            <tbody>
                <c:forEach var="message" items="${guestbook.messages}">
                    <tr>
                        <td>${message.name}</td>
                        <td>${message.email}</td>
                        <td>${message.msg}</td>
                    </tr>
                </c:forEach>
            </tbody>
        </table>
    </body>
</html>

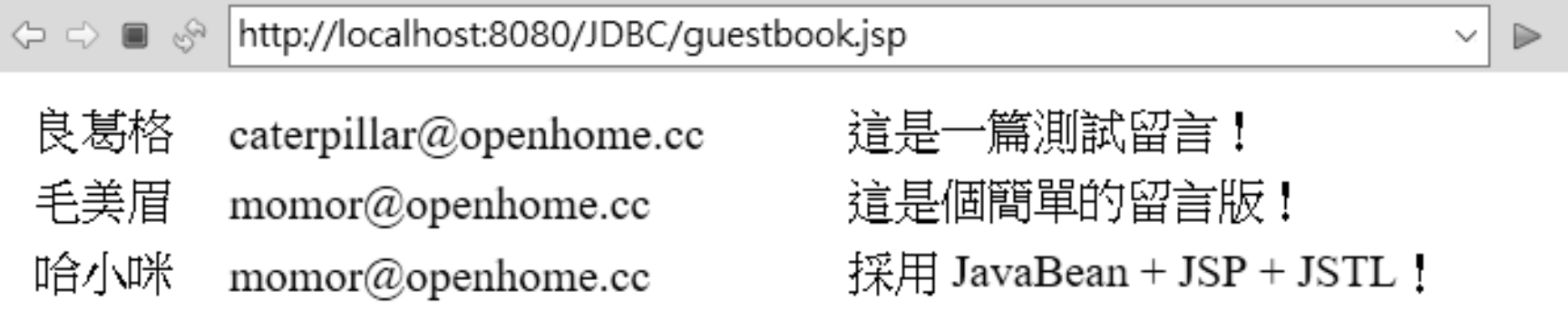
```

↑ 呼叫
getMessages()
方法取得留言

```

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>
<c:set target="${pageContext.request}"
    property="characterEncoding" value="UTF-8"/> ← 設定請求編碼處理
<jsp:useBean id="guestbook" ← 使用 GuestBookBean 方式為 UTF-8
    class="cc.openhome.GuestBookBean" scope="application"/>
<c:if test="${param.msg != null}"> ← 如果是要新增留言的話
    <jsp:useBean id="newMessage" class="cc.openhome.Message"/>
    <jsp:setProperty name="newMessage" property="*"/>
    <c:set target="${guestbook}" ← 呼叫 setMessage() 方法新增留言
        property="messages" value="${newMessage}"/>

```



```

cellpadding="2" cellspacing="2">
<tbody>
    <c:forEach var="message" items="${guestbook.messages}">
        <tr>
            <td>${message.name}</td>
            <td>${message.email}</td>
            <td>${message.msg}</td>
        </tr>
    </c:forEach>
</tbody>
</table>
</body>
</html>

```

↑ 呼叫
getMessages()
方法取得留言

使用PreparedStatement

```
statement.executeUpdate(  
    "INSERT INTO t_message(name, email, msg) VALUES ('"  
    + message.getName() + "', '"  
    + message.getEmail() + "', '"  
    + message.getMsg() + "')");  
  
PreparedStatement stmt = conn.prepareStatement(  
    "INSERT INTO t_message VALUES (?, ?, ?, ?)");  
  
stmt.setInt(1, 2);  
stmt.setString(2, "momor");  
stmt.setString(3, "momor@mail.com");  
stmt.setString(4, "message2...");  
stmt.executeUpdate();  
stmt.clearParameters();
```

- 使用PreparedStatement改寫先前GuestBookBean中setMessage()

```
public void setMessage(Message message) {  
    try(Connection conn = DriverManager.getConnection(  
                                                jdbcUri, username, password);  
        PreparedStatement statement = conn.prepareStatement(  
            "INSERT INTO t_message(name, email, msg) VALUES (?, ?, ?)")) {  
        statement.setString(1, message.getName());  
        statement.setString(2, message.getEmail());  
        statement.setString(3, message.getMsg());  
        statement.executeUpdate();  
    } catch (SQLException e) {  
        throw new RuntimeException(e);  
    }  
}
```


使用PreparedStatement

- 安全 ...

```
Statement statement = connection.createStatement();
String queryString = "SELECT * FROM user_table WHERE username='" +
    username + "' AND password='" + password + "'";
ResultSet resultSet = statement.executeQuery(queryString);
```

```
SELECT * FROM user_table
WHERE username='caterpillar' AND password='123456'
```

```
SELECT * FROM user_table
WHERE username='caterpillar' AND password=' ' OR '1'='1'
```

```
SELECT * FROM user_table
WHERE username='caterpillar' AND password='' OR '1'='1'
```

使用PreparedStatement

- 以下username與password將被視作是SQL中純粹的字串，而不會當作SQL語法來解釋

```
PreparedStatement stmt = conn.prepareStatement(  
    "SELECT * FROM user_table WHERE username=? AND password=?");  
stmt.setString(1, username);  
stmt.setString(2, password);
```

使用CallableStatement

- 呼叫資料庫的預存程序（Stored Procedure）

```
{?= call <程序名稱>[<引數 1>,<引數 2>, ...]}  
{call <程序名稱>[<引數 1>,<引數 2>, ...]}
```

- 必須呼叫**prepareCall()**建立
CallableStatement實例

Java型態與SQL型態對應

Java 型態	SQL 型態
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
float	FLOAT
double	DOUBLE
byte[]	BINARY、VARBINARY、LONGBINARY
java.lang.String	CHAR、VARCHAR、LONGVARCHAR
java.math.BigDecimal	NUMERIC、DECIMAL
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP

Java型態與SQL型態對應

- 日期時間在JDBC中，並不是使用 `java.util.Date`
 - 年、月、日、時、分、秒、毫秒
- 在JDBC中要表示日期，是使用 `java.sql.Date`
 - 年、月、日
- 要表示時間的話則是使用 `java.sql.Time`
 - 時、分、秒
- 使用 `java.sql.Timestamp`
 - 時、分、秒、微秒

使用DataSource取得連線

- 取得連線等與資料庫來源相關的行為規範在 **javax.sql.DataSource** 介面
- 可以透過JNDI（Java Naming Directory Interface）來取得所需的資源物件

使用DataSource取得連線

- 如果是在Web應用程式中想要獲得DataSource實例

```
try {  
    Context initContext = new InitialContext();  
    Context envContext = (Context) initContext.lookup("java:/comp/env");  
    dataSource = (DataSource) envContext.lookup("jdbc/demo");  
} catch (NamingException ex) {  
    ...  
}
```

```
public class DatabaseBean implements Serializable {
    private DataSource dataSource;

    public DatabaseBean() {
        try {
            Context initContext = new InitialContext();
            Context envContext = (Context)
                initContext.lookup("java:/comp/env");
            dataSource = (DataSource) envContext.lookup("jdbc/demo");
        } catch (NamingException ex) {
            throw new RuntimeException(ex);
        }
    }

    public boolean isConnectedOK() {
        try(Connection conn = dataSource.getConnection()) {
            return !conn.isClosed();
        } catch (SQLException e) {
            throw new RuntimeException(e);
        }
    }
}
```

↑ 查找 jdbc/demo 對應
的 DataSource 物件

透過 DataSource 物件取得連線



使用DataSource取得連線

- 在web.xml中設定

```
</web-app ...>
  // 略...
  <resource-ref>
    <res-ref-name>jdbc/demo</res-ref-name>
    <res-type>javax.sql.DataSource</res-type>
    <res-auth>Container</res-auth>
    <res-sharing-scope>Shareable</res-sharing-scope>
  </resource-ref>
</web-app>
```

使用ResultSet捲動、更新資料

- 從JDBC 2.0開始，ResultSet並不僅可以使用**previous()**、**first()**、**last()**等方法前後移動資料游標，還可以呼叫updateXXX()、updateRow()等方法進行資料修改

使用ResultSet捲動、更新資料

- 建立Statement或PreparedStatement實例時，可以指定結果集類型與並行方式

```
createStatement(int resultSetType, int resultSetConcurrency)  
prepareStatement(String sql,  
                  int resultSetType, int resultSetConcurrency)
```

- 結果集類型可以指定三種設定：
 - ResultSet.TYPE_FORWARD_ONLY (預設)
 - ResultSet.TYPE_SCROLL_INSENSITIVE
 - ResultSet.TYPE_SCROLL_SENSITIVE

使用ResultSet捲動、更新資料

- 更新設定可以有兩種指定：
 - ResultSet.CONCUR_READ_ONLY (預設)
 - ResultSet.CONCUR_UPDATABLE

```
Statement stmt = conn.createStatement(  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATEABLE);
```

```
PreparedStatement stmt = conn.prepareStatement(  
    "SELECT * FROM t_message",  
    ResultSet.TYPE_SCROLL_INSENSITIVE,  
    ResultSet.CONCUR_UPDATEABLE);
```

使用ResultSet捲動、更新資料

- 資料游標移動...

```
Statement stmt = conn.createStatement("SELECT * FROM t_message",
                                       ResultSet.TYPE_SCROLL_INSENSITIVE,
                                       ResultSet.CONCUR_READ_ONLY);

ResultSet rs = stmt.executeQuery();
rs.absolute(2);           // 移至第 2 列
rs.next();               // 移至第 3 列
rs.first();              // 移至第 1 列
boolean b1 = rs.isFirst(); // b1 是 true
```

使用ResultSet捲動、更新資料

- 使用ResultSet進行資料修改
 - 必須選取單一表格
 - 必須選取主鍵
 - 必須選取所有NOT NULL的值
- 更新資料...

```
Statement stmt = conn.prepareStatement("SELECT * FROM t_message",
                                         ResultSet.TYPE_SCROLL_INSENSITIVE,
                                         ResultSet.CONCUR_UPDATABLE);

ResultSet rs = stmt.executeQuery();
rs.next();
rs.updateString(3, "caterpillar@openhome.cc");
rs.updateRow();
```

使用ResultSet捲動、更新資料

- 新增資料 ...

```
Statement stmt = conn.prepareStatement("SELECT * FROM t_message",
                                         ResultSet.TYPE_SCROLL_INSENSITIVE,
                                         ResultSet.CONCUR_UPDATABLE);

ResultSet rs = stmt.executeQuery();
rs.moveToInsertRow();
rs.updateString(2, "momor");
rs.updateString(3, "momor@openhome.cc");
rs.updateString(4, "blah..blah");
rs.insertRow();
rs.moveToCurrentRow();
```

使用ResultSet捲動、更新資料

- 刪除資料 ...

```
Statement stmt = conn.prepareStatement("SELECT * FROM t_message",
                                         ResultSet.TYPE_SCROLL_INSENSITIVE,
                                         ResultSet.CONCUR_UPDATABLE);

ResultSet rs = stmt.executeQuery();
rs.absolute(3);
rs.deleteRow();
```


批次更新

- 以下每一次執行executeUpdate()，其實都會向資料庫發送一次SQL

```
Statement stmt = conn.createStatement();  
while(someCondition) {  
    stmt.executeUpdate(  
        "INSERT INTO t_message(name,email,msg) VALUES ('...', '...', '...')");  
}
```

批次更新

- 可以使用**addBatch()** 方法來收集SQL
- 使用**executeBatch()** 方法將所收集的SQL傳送出去

```
Statement stmt = conn.createStatement();
while(someCondition) {
    stmt.addBatch(
        "INSERT INTO t_message(name,email,msg) VALUES('...', '...', '...')");
}
stmt.executeBatch();
```

批次更新

- 使用`executeBatch()`時，SQL的執行順序，就是`addBatch()`時的順序
- `executeBatch()`會傳回`int[]`，代表每筆SQL造成的資料異動列數
- 先前已開啟的`ResultSet`會被關閉，執行過後收集SQL用的`List`會被清空
- 任何的SQL錯誤，會丟出**`BatchUpdateException`**
- 可以使用這個物件的**`getUpdateCounts()`**取得`int[]`，代表先前執行成功的SQL所造成的異動筆數

批次更新

- PreparedStatement使用批次更新

```
PreparedStatement stmt = conn.prepareStatement(
    "INSERT INTO t_message(name,email,msg) VALUES(?, ?, ?)");
while(someCondition) {
    stmt.setString(1, "..");
    stmt.setString(2, "..");
    stmt.setString(3, "..");
    stmt.addBatch(); // 收集參數
}
stmt.executeBatch(); // 送出所有參數
```

Blob與Clob

- BLOB全名Binary Large Object，用於儲存大量的二進位資料，像是圖檔、影音檔等
- CLOB全名Character Large Object，用於儲存大量的文字資料
- **java.sql.Blob**與**java.sql.Clob**兩個類別分別代表BLOB與CLOB資料

Blob與Clob

- Blob擁有`getBinaryStream()`、`getBytes()`等方法，可以取得代表欄位來源的`InputStream`或欄位的`byte[]`資料
- Clob擁有`getCharacterStream()`、`getAsciiStream()`等方法，可以取得`Reader`或`InputStream`等資料

Blob與Clob

- 也可以把BLOB欄位對應byte[]或輸入/輸出串流
- 使用PreparedStatement的**setBytes()**來設定要存入的byte[]資料，使用**setBinaryStream()**來設定代表輸入來源的InputStream
- 使用ResultSet的**getBytes()**以byte[]取得欄位中儲存的資料，或以**getBinaryStream()**取得代表欄位來源的InputStream

Blob與Clob

```
InputStream in = readFileAsInputStream(".....");
PreparedStatement stmt = conn.prepareStatement(
    "INSERT INTO IMAGES(src, img) VALUE(?, ?)");
stmt.setString(1, "...");
stmt.setBinaryStream(2, in);
stmt.executeUpdate();
```

```
PreparedStatement stmt = conn.prepareStatement(
    "SELECT img FROM IMAGES");
ResultSet rs = stmt.executeQuery();
while(rs.next()) {
    InputStream in = rs.getBinaryStream(1);
    //..使用 InputStream 作資料讀取
}
```


實例

http://localhost:8080/JDBC/file.jsp

選取檔案： 瀏覽...

上傳

檔案名稱	上傳時間	操作
caterpillar.jpg	2018-02-13T13:58:31.296	下載 / 刪除
ServletJSP改版.txt	2018-02-13T14:00:01.089	下載 / 刪除

檔案下載

您要開啟或儲存這個檔案？



名稱: ServletJSP改版.txt

類型: 應用程式

從: localhost

開啟舊檔(O)

儲存(S)

取消



來自網際網路的檔案或許有用，但有些檔案可能會危害您的電腦。若不信任此來源，請勿開啟或儲存此檔案。[有什麼風險？](#)

簡介交易

- 原子性（**A**tomicity）
- 一致性（**C**onsistency）
- 隔離行為（**I**solation behavior）
- 持續性（**D**urability）

```
Connection conn = null;
try {
    conn = dataSource.getConnection();
    conn.setAutoCommit(false); // 取消自動提交
    Statement stmt = conn.createStatement();
    stmt.executeUpdate("INSERT INTO ...");
    stmt.executeUpdate("INSERT INTO ...");
    conn.commit(); // 提交
}
catch(SQLException e) {
    e.printStackTrace();
    if(conn != null) {
        try {
            conn.rollback(); // 撤回
        }
        catch(SQLException ex) {
            ex.printStackTrace();
        }
    }
}
finally {
    ...
    if(conn != null) {
        try {
            conn.setAutoCommit(true); // 回復自動提交
            conn.close();
        }
        catch(SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```

```
conn.setAutoCommit(false);
Statement stmt = conn.createStatement();
stmt.executeUpdate("INSERT INTO ...");
...
point = conn.setSavepoint(); // 設定儲存點
stmt.executeUpdate("INSERT INTO ...");
...
conn.commit();
}
catch(SQLException e) {
    e.printStackTrace();
    if(conn != null) {
        try {
            if(point == null) {
                conn.rollback();
            }
            else {
                conn.rollback(point); // 撤回儲存點
                conn.releaseSavepoint(point); // 釋放儲存點
            }
        }
        catch(SQLException ex) {
            ex.printStackTrace();
        }
    }
}
finally {
    ...
    if(conn != null) {
        try {
            conn.setAutoCommit(true);
```

```
try {
    conn.setAutoCommit(false);
    stmt = conn.createStatement();
    while(someCondition) {
        stmt.addBatch("INSERT INTO ...");
    }
    stmt.executeBatch();
    conn.commit();
} catch(SQLException ex) {
    ex.printStackTrace();
    if(conn != null) {
        try {
            conn.rollback();
        } catch(SQLException e) {
            e.printStackTrace();
        }
    }
} finally {
    ...
    if(conn != null) {
        try {
            conn.setAutoCommit(true);
            conn.close();
        }
        catch(SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```

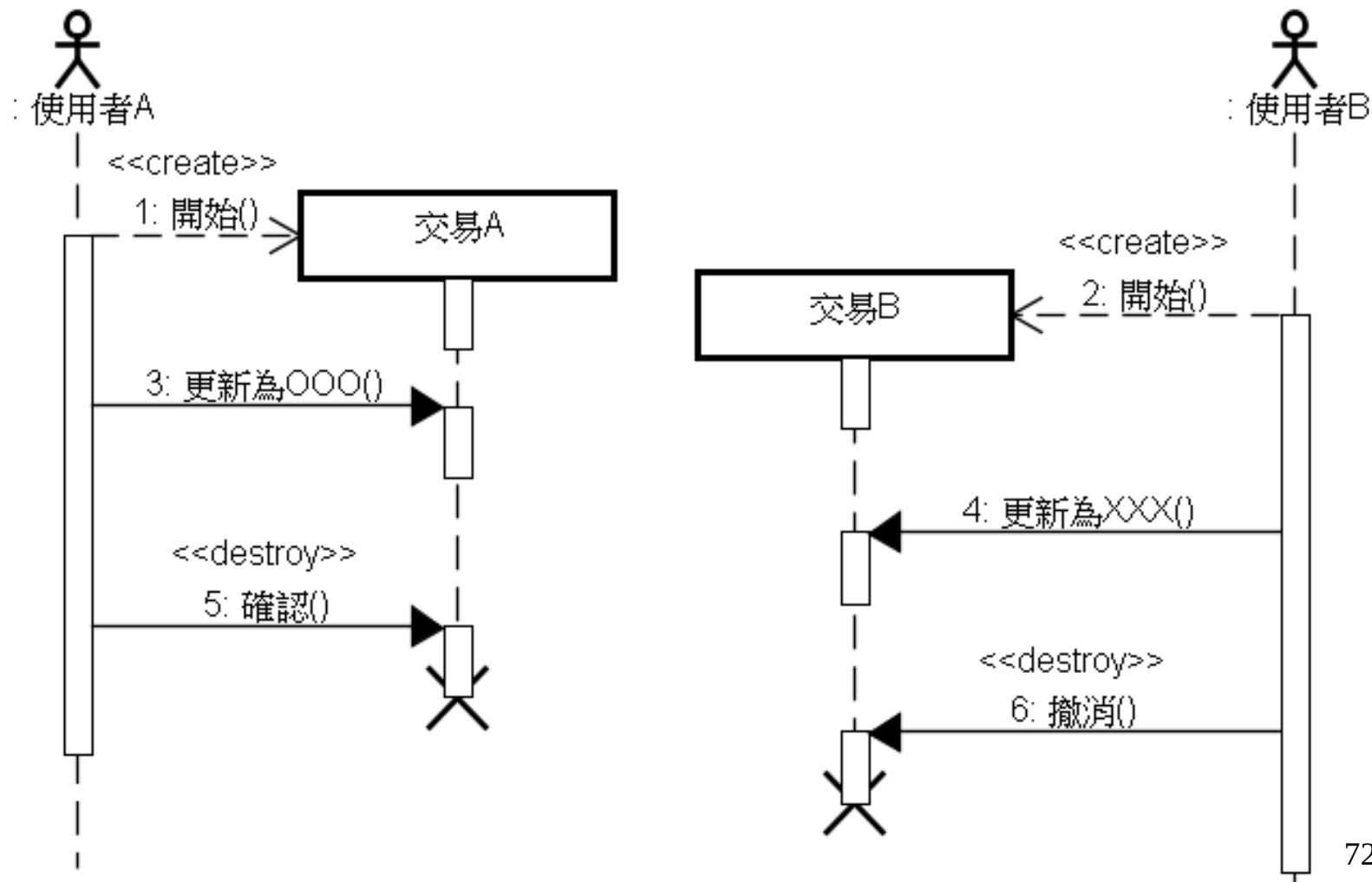
隔離行為

- 可以透過Connection的
getTransactionIsolation() 取得資料庫目前的隔離行為設定
- 透過**setTransactionIsolation()** 可提示資料庫設定指定的隔離行為

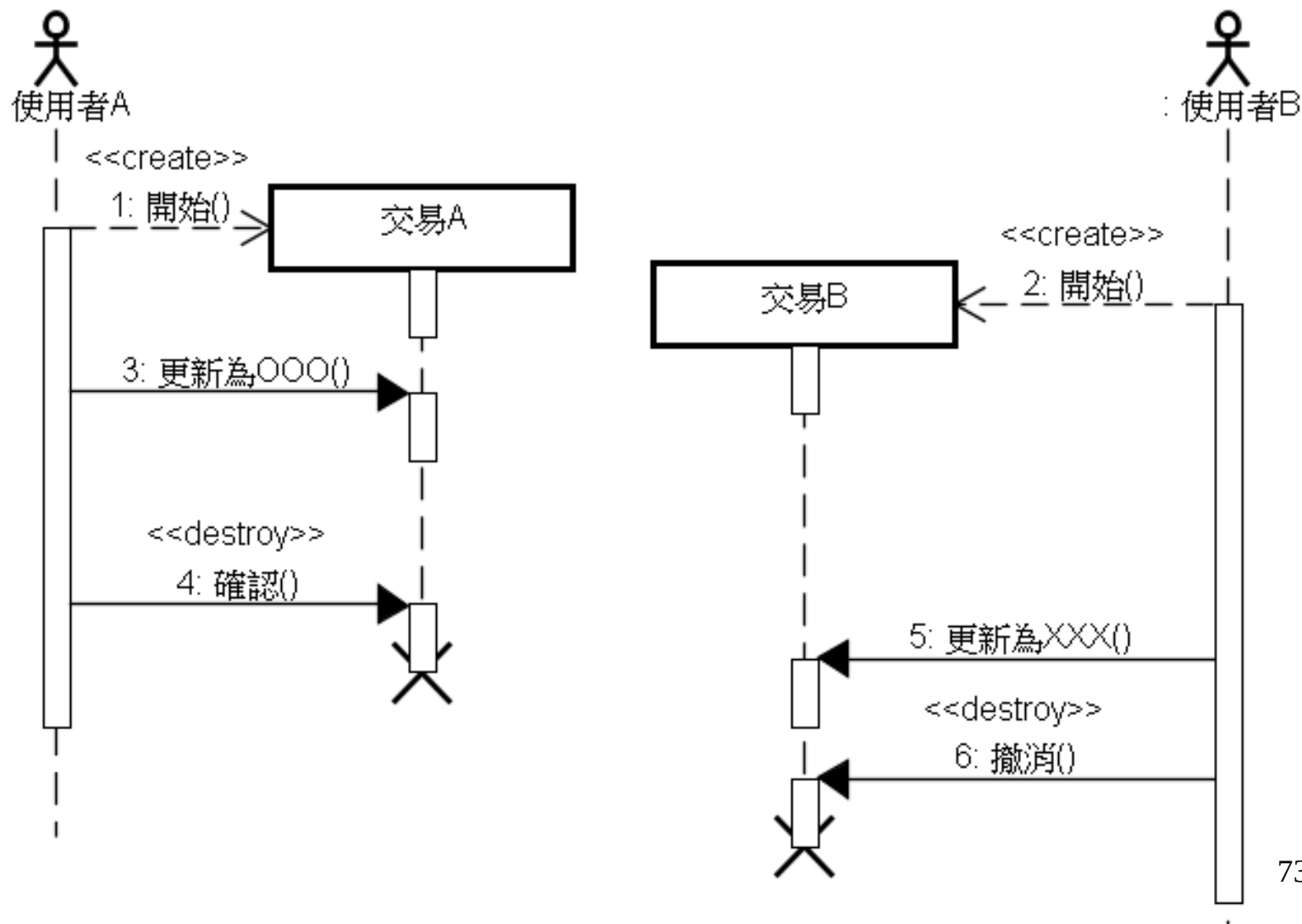
隔離行為

- 可設定常數是定義在Connection上
 - TRANSACTION_NONE
 - TRANSACTION_UNCOMMITTED
 - TRANSACTION_COMMITTED
 - TRANSACTION_REPEATABLE_READ
 - TRANSACTION_SERIALIZABLE

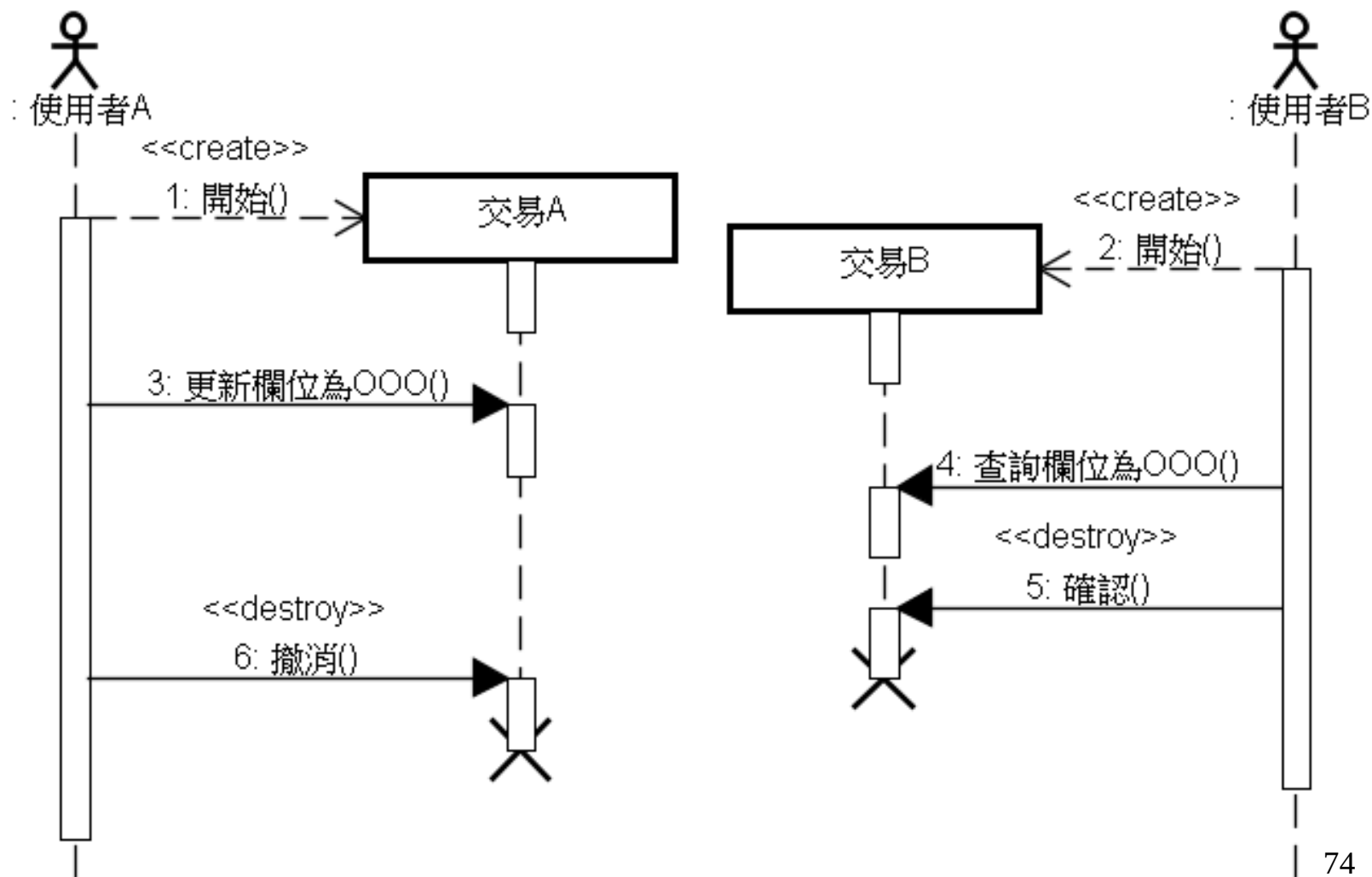
更新遺失 (Lost update)



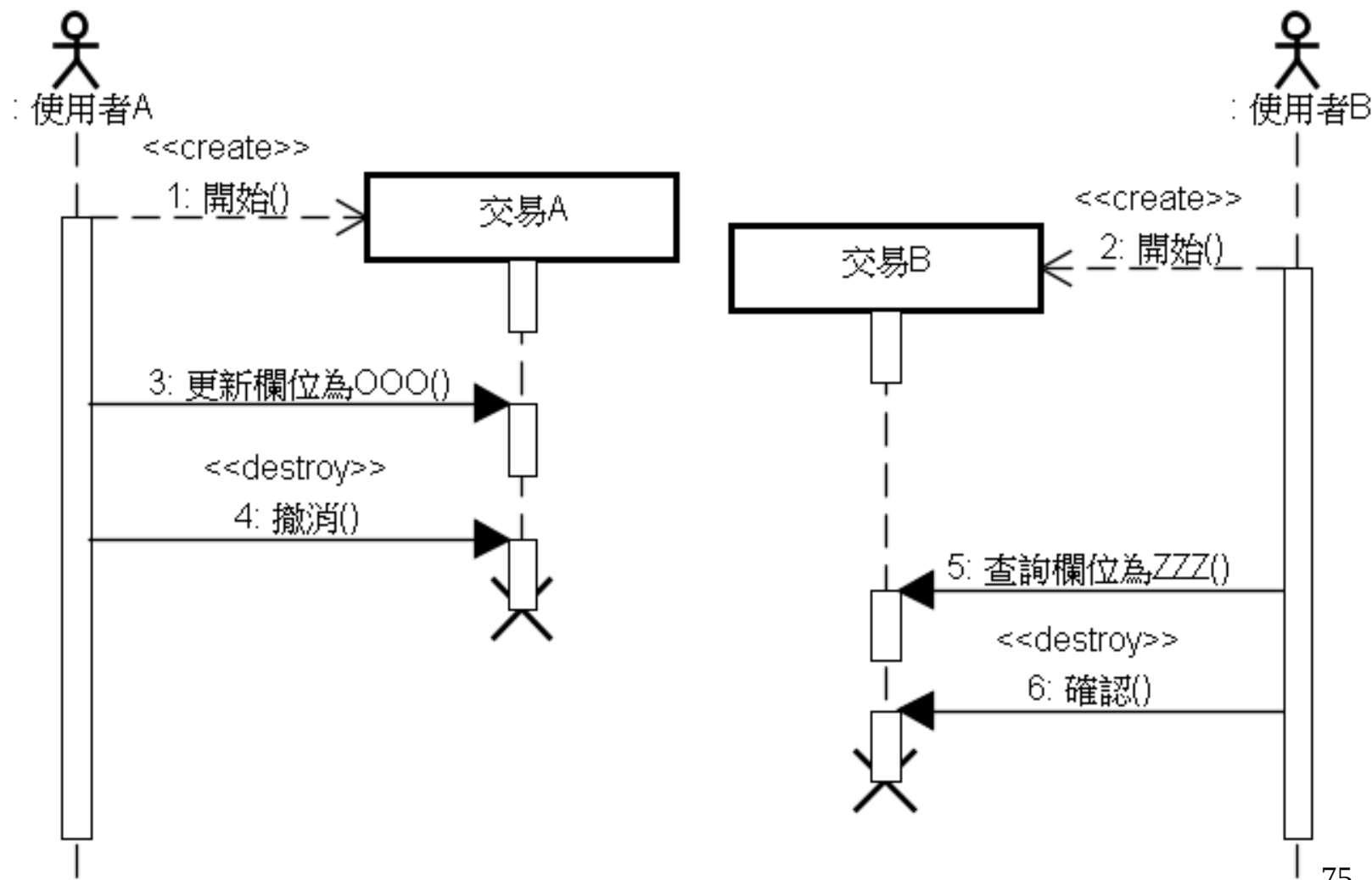
Read uncommitted



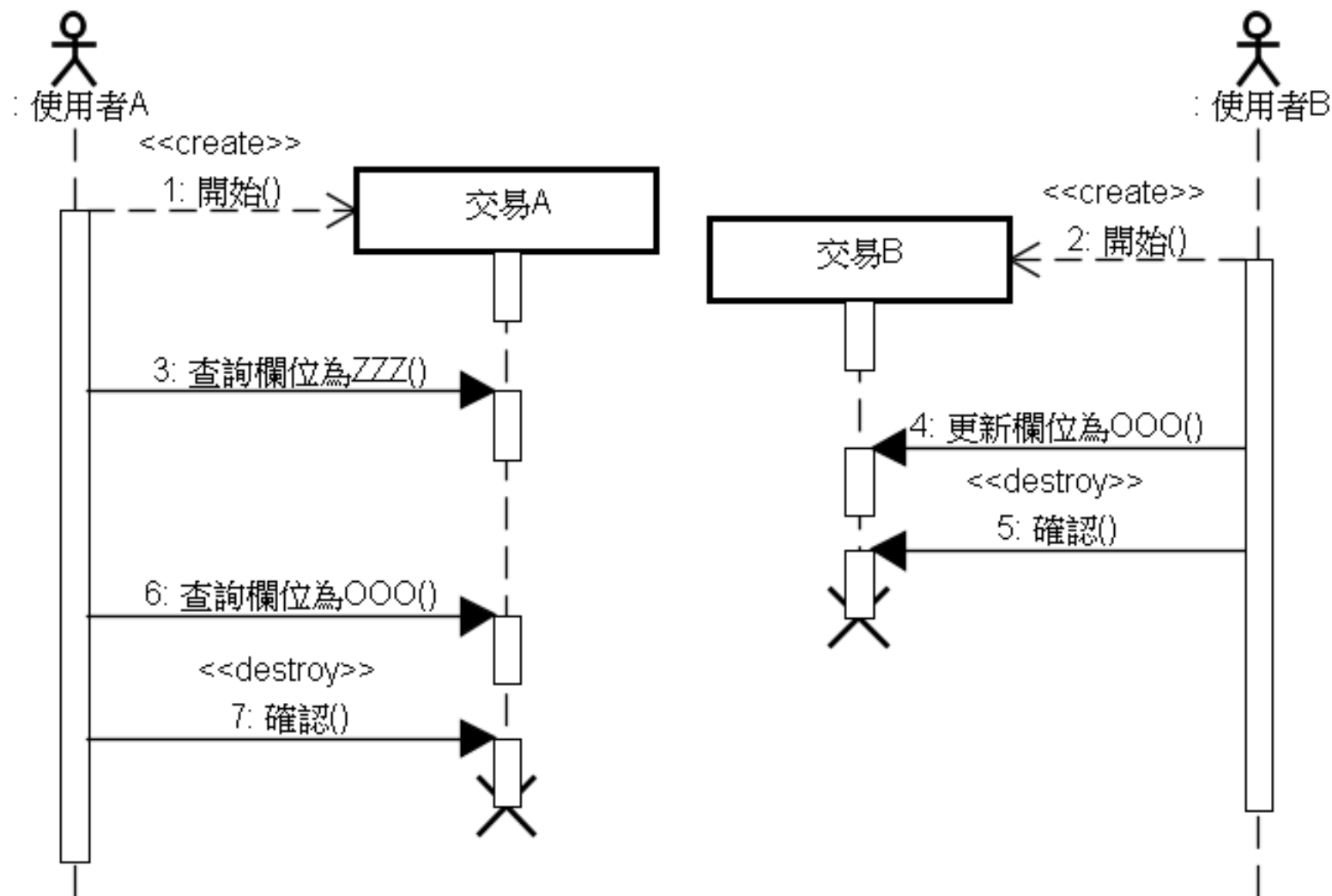
髒讀 (Dirty read)



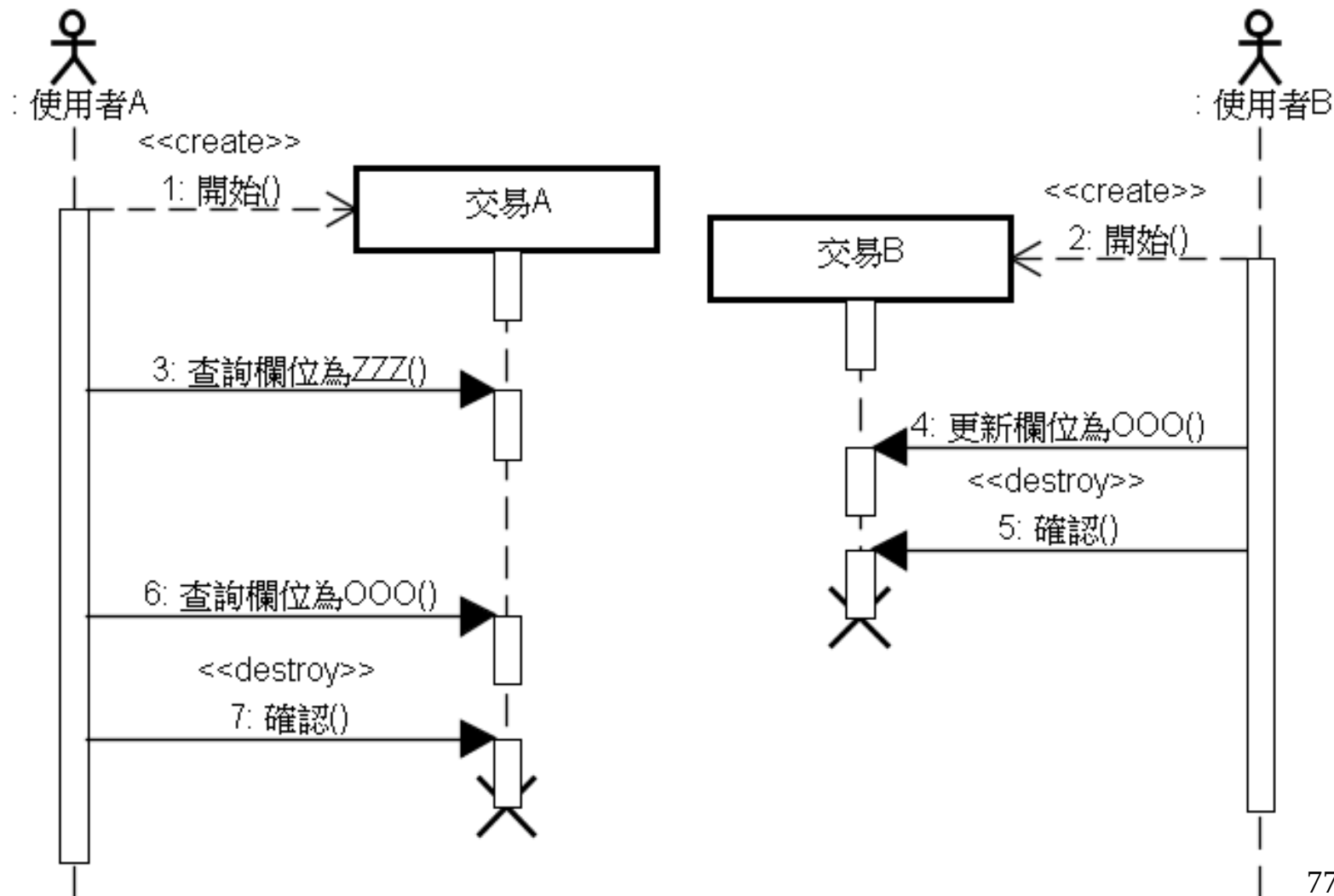
Read committed



無法重複的讀取 (Unrepeatable read)



Repeatable read



幻讀 (Phantom read)

- 同一交易期間，讀取到的資料筆數不一致。例如交易A第一次讀取得到五筆資料，此時交易B新增了一筆資料，導致交易B再次讀取得到六筆資料
- 如果隔離行為設定為可重複讀取，但發生幻讀現象，可以設定隔離層級為「可循序」 (Serializable)，也就是在有交易時若有資料不一致的疑慮，交易必須可以照順序逐一進行

隔離行為與可預防之問題

隔離行為	Lost update	Dirty read	Unrepeatable read	Phantom read
Read uncommitted	預防			
Read committed	預防	預防		
Repeatable read	預防	預防	預防	
Serializable	預防	預防	預防	預防

隔離行為

- 得知是否支援某個隔離行為

```
DatabaseMetadata meta = conn.getMetaData();  
boolean isSupported = meta.supportsTransactionIsolationLevel(  
    Connection.TRANSACTION_READ_COMMITTED);
```


簡介metadata

- 詮讀資料的資料（Data about data）
- 可以透過Connection的**getMetaData()**方法取得**DatabaseMetaData**物件
- 可以透過ResultSet的getMetaData()方法，取得ResultSetMetaData物件

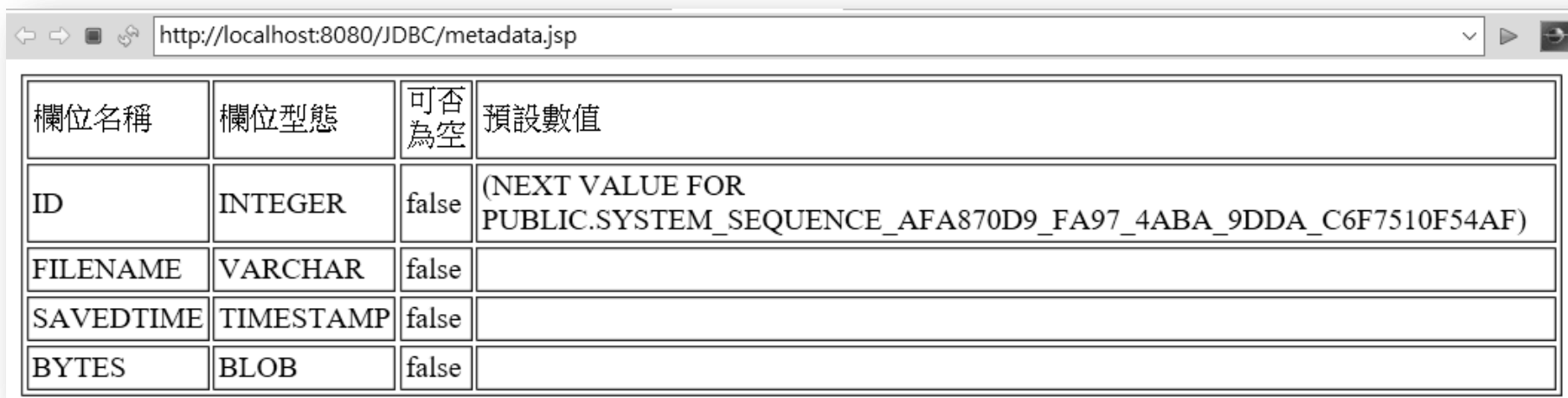
```
public List<ColumnInfo> getAllColumnInfo() {
    Connection conn = null;
    ResultSet crs = null;
    SQLException ex = null;
    List<ColumnInfo> infos = null;
    try {
        conn = dataSource.getConnection();
        DatabaseMetaData meta = conn.getMetaData();
        crs = meta.getColumns(
            "demo", null, "t_files", null);
        infos = new ArrayList<ColumnInfo>();
        while(crs.next()) {
            ColumnInfo info = new ColumnInfo();
            info.setName(crs.getString("COLUMN_NAME"));
            info.setType(crs.getString("TYPE_NAME"));
            info.setSize(crs.getInt("COLUMN_SIZE"));
            info.setNullable(crs.getBoolean("IS_NULLABLE"));
            info.setDef(crs.getString("COLUMN_DEF"));
            infos.add(info);
        }
    } catch (SQLException e) {
        ex = e;
    }
}
```

❶ 查詢 t_files 表格所有欄位

❷ 用來收集欄位資訊

❸ 封裝欄位名稱、型態、大小、可否為空、預設值

簡介metadata

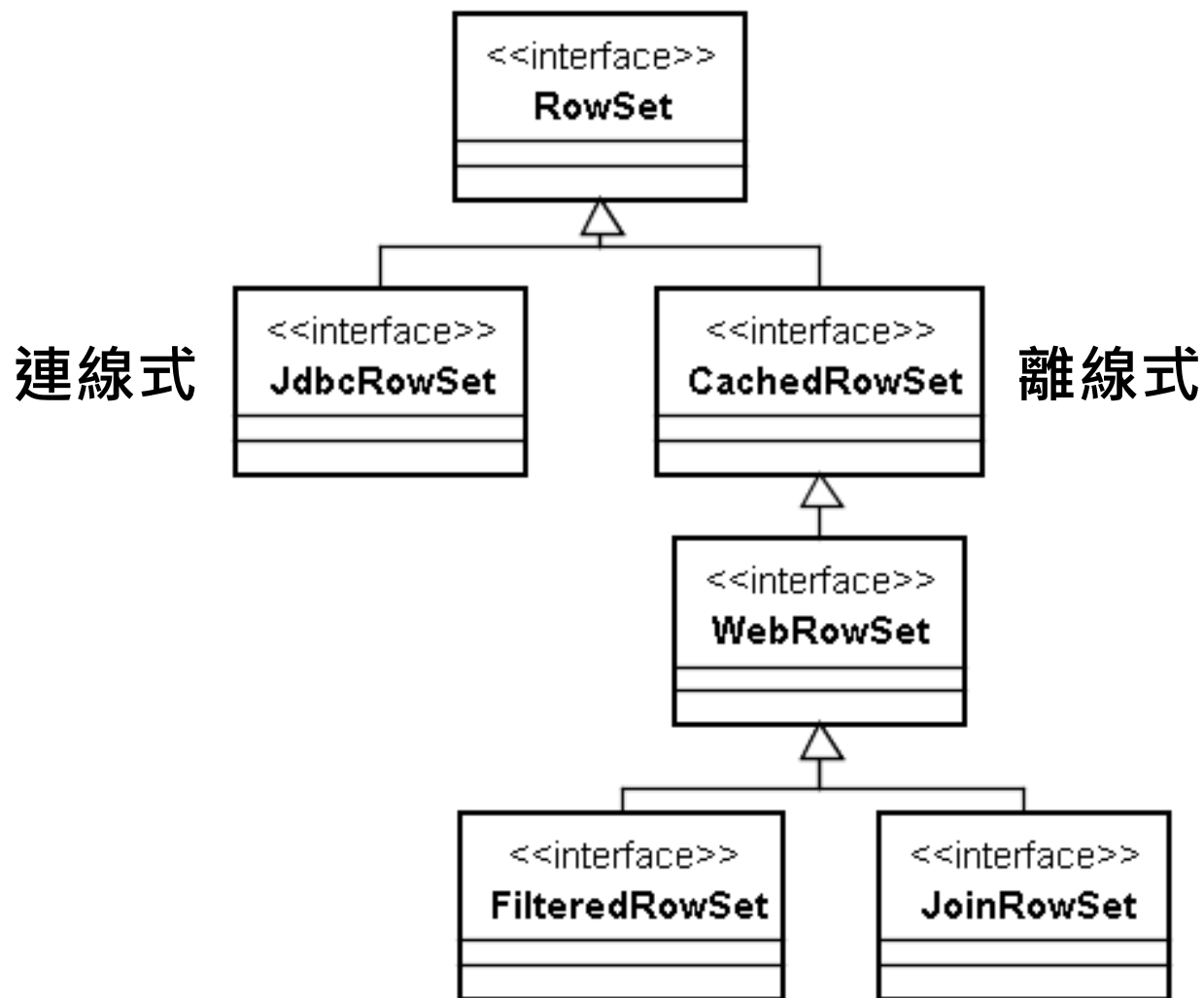


欄位名稱	欄位型態	可否為空	預設數值
ID	INTEGER	false	(NEXT VALUE FOR PUBLIC.SYSTEM_SEQUENCE_AFA870D9_FA97_4ABA_9DDA_C6F7510F54AF)
FILENAME	VARCHAR	false	
SAVEDTIME	TIMESTAMP	false	
BYTES	BLOB	false	

簡介RowSet

- **javax.sql.RowSet**代表資料的列集合
 - 這邊的資料並不一定是資料庫中的資料，可以是試算表資料、XML資料或任何具有列集合概念的資料來源
- 是ResultSet的子介面，新增了一些行為，像是透過setCommand()設定查詢指令、透過execute()執行查詢指令以填充資料等

簡介RowSet



簡介RowSet

- JdbcRowSet是連線式的RowSet
 - 操作JdbcRowSet期間，會保持與資料庫的連線，可視為取得、操作ResultSet行為封裝，可簡化JDBC程式的撰寫，或作為JavaBean使用
- CachedRowSet為離線式的RowSet
 - 查詢並充填完資料後，就會斷開與資料來源的連線，而不用佔據相關連線資源，必要也可以再與資料來源連線進行資料同步

簡介RowSet

- 使用RowSet查詢資料

```
JdbcRowSet rowset = new JdbcRowSetImpl();  
rowset.setUrl("jdbc:h2:tcp://localhost/c:/workspace/JDBC/demo");  
rowset.setUsername("caterpillar");  
rowset.setPassword("12345678");  
rowset.setCommand("SELECT * FROM t_messages WHERE id = ?");  
rowset.setInt(1, 1);  
rowset.execute();
```

```
private JdbcRowSet rowset;
public GuestBookBean2() throws SQLException {
    rowset = new JdbcRowSetImpl();
    rowset.setDataSourceName("java:/comp/env/jdbc/demo");
}

private void loadTable() throws SQLException {
    rowset.setCommand("SELECT * FROM t_message");
    rowset.execute();
}

public void setMessage(Message message) throws SQLException {
    loadTable();
    rowset.moveToInsertRow();
    rowset.updateString(2, message.getName());
    rowset.updateString(3, message.getEmail());
    rowset.updateString(4, message.getMsg());
    rowset.insertRow();
}

public List<Message> getMessages() throws SQLException {
    loadTable();
    List<Message> messages = new ArrayList<>();
    rowset.beforeFirst();
    while (rowset.next()) {
        Message message = new Message();
        message.setId(rowset.getLong(1));
        message.setName(rowset.getString(2));
        message.setEmail(rowset.getString(3));
        message.setMsg(rowset.getString(4));
        messages.add(message);
    }
    return messages;
}

@Override
protected void finalize() throws Throwable {
    if(rowset != null) {
        rowset.close();
    }
}
```


簡介RowSet

- 在查詢之後，想要離線進行操作，則可以使用CachedRowSet或其子介面實作物件
- 使用**close()**關閉CachedRowSet，若在相關更新操作之後，想與再與資料來源進行同步，則可以呼叫**acceptChanges()**方法

```
conn.setAutoCommit(false); // conn 是 Connection  
rowSet.acceptChanges(conn); // rowSet 是 CachedRowSet  
conn.setAutoCommit(true);
```

簡介RowSet

- **WebRowSet**是CachedRowSet的子介面，其不僅具備離線操作，還能進行XML讀寫

```
protected void doGet(HttpServletRequest request,
                    HttpServletResponse response)
                    throws ServletException, IOException {
    response.setContentType("text/xml;charset=UTF-8");
    try {
        rowset.writeXml(response.getOutputStream());
    } catch (SQLException e) {
        throw new ServletException(e);
    }
}
```

簡介RowSet

```
http://localhost:8080/JDBC/xmlMessage
- <data>
  - <currentRow>
    <columnValue>1</columnValue>
    <columnValue>良葛格</columnValue>
    <columnValue>caterpillar@openhome.cc</columnValue>
    <columnValue>這是一篇測試留言！</columnValue>
  </currentRow>
  - <currentRow>
    <columnValue>2</columnValue>
    <columnValue>毛美眉</columnValue>
    <columnValue>momor@openhome.cc</columnValue>
    <columnValue>這是簡單的留言版！</columnValue>
  </currentRow>
  - <currentRow>
    <columnValue>3</columnValue>
    <columnValue>哈小咪</columnValue>
    <columnValue>hamimi@openhome.cc</columnValue>
    <columnValue>採用 JavaBean + JSP + JSTL ! </columnValue>
  </currentRow>
</data>
```

```
<body>
  <c:import var="xml" url="xmlMessage" charEncoding="UTF-8" />
  <c:set var='xmlns'>
    xmlns="http://java.sun.com/xml/ns/jdbc"
  </c:set>

  <!-- JSTL 1.1 不支援 XML Namespace，所以用空字串取代掉 -->
  <x:parse var="webRowSet" doc="{fn:replace(xml, xmlns, '')}"/>
  <h2>友站的留言</h2>
  <table border="1">
    <tr bgcolor="#00ff00">
      <th align="left">名稱</th>
      <th align="left">郵件</th>
      <th align="left">留言</th>
    </tr>
    <x:forEach var="row" select="$webRowSet//currentRow">
      <tr>
        <td><x:out select="$row/columnValue[2]"/></td>
        <td><x:out select="$row/columnValue[3]"/></td>
        <td><x:out select="$row/columnValue[4]"/></td>
      </tr>
    </x:forEach>
  </table>
</body>
```

```
<body>
```

```
  <c:import var="xml" url="xmlMessage" charEncoding="UTF-8" />
```

```
  <c:set var='xmlns'
```

```
    xmlns="http://java.sun.com/xml/ns/jdbc"
```

```
</c:set>
```

```
<!-- http://localhost:8080/JDBC/xmlMessage.jsp -->
```

```
<h1>
```

```
<h2>友站的留言
```

```
<table>
```

名稱	郵件	留言
良葛格	caterpillar@openhome.cc	這是一篇測試留言！
毛美眉	momor@openhome.cc	這是簡單的留言版！
哈小咪	hamimi@openhome.cc	採用 JavaBean + JSP + JSTL！

```
  <td><x:out select="$row/columnValue[2]" /></td>
```

```
  <td><x:out select="$row/columnValue[3]" /></td>
```

```
  <td><x:out select="$row/columnValue[4]" /></td>
```

```
</tr>
```

```
</x:forEach>
```

```
</table>
```

```
</body>
```

簡介RowSet

- **FilteredRowSet**可以對列集合進行過濾，實現類似SQL中WHERE等條件式的功能
- **JoinRowSet**則可以讓你結合兩個RowSet物件，實現類似SQL中JOIN的功能
- docs.oracle.com/javase/tutorial/jdbc/basics/gettingstarted.html

使用SQL標籤庫

```
<%@taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql"%>
```

```
<sql:setDataSource dataSource="java:/comp/env/jdbc/demo"/>
```

```
<sql:setDataSource driver="org.h2.Driver"
                    user="caterpillar"
                    password="12345678"
                    url="jdbc:h2:tcp://localhost/c:/workspace/JDBC/demo"/>
```

```
<sql:query sql="SELECT * FROM t_message" var="messages"/>
```

```
<sql:query dataSource="${dataSource}" var="messages">
    SELECT * FROM t_message
</sql:query>
```

使用SQL標籤庫

- `<sql:query>`的查詢結果是**Result**型態，具有**`getColumnNames()`**、**`getRowCount()`**、**`getRows()`**等方法

```
<sql:query sql="SELECT * FROM t_message" var="messages"/>
<c:forEach var="message" items="${messages.rows}">
    ${message.name}<br>
    ${message.email}<br>
    ${message.msg}
</c:forEach>
```


使用SQL標籤庫

- Result也有**getRowsByIdex()**方法，可以Object[][]傳回查詢資料

```
<sql:query sql="SELECT * FROM t_message" var="messages"/>
<c:forEach var="message" items="${messages.rowsByIndex}">
    ${message[0]}<br>
    ${message[1]}<br>
    ${message[2]}
</c:forEach>
```

使用SQL標籤庫

- 在資料庫中新增一筆資料

```
<sql:update>
    INSERT INTO t_message(name, email, msg)
        VALUES('Justin', 'caterpillar@openhome.cc', 'This is a test!')
</sql:update>
```

- 以下寫法雖可以但不建議

```
<sql:update>
    INSERT INTO t_message(name, email, msg)
        VALUES(${param.user}, ${param.email}, ${param.msg})
</sql:update>
```

使用SQL標籤庫

- 可以使用在SQL中使用佔位字元，並搭配 **<sql:param>** 標籤來設定佔位字元的值

```
<sql:update>
    INSERT INTO t_message(name, email, msg) VALUES(?, ?, ?)
    <sql:param value="${param.name}"/>
    <sql:param value="${param.email}"/>
    <sql:param value="${param.msg}"/>
</sql:update>
```

使用SQL標籤庫

- 如果欄位是日期時間格式，則可以使用 **<sql:paramDate>** 標籤，可以透過 **type** 屬性設定，指定使用“time”、“date”或“timestamp”的值
- **<sql:param>**、**<sql:paramDate>** 也可以搭配 **<sql:query>** 使用。

使用SQL標籤庫

- 如果有必要指定交易隔離行為，則可以透過 **<sql:transaction>** 標籤指定
- 設定isolation屬性為"read_uncommitted"、"read_committed"、"repeatable"或"serializable"

```
<%@taglib prefix="sql" uri="http://java.sun.com/jsp/jstl/sql"%>
<sql:setDataSource dataSource="jdbc/demo"/>
<c:set target="${pageContext.request}"
      property="characterEncoding" value="UTF-8"/>
<c:if test="${param.msg != null}">
  <sql:update>
    INSERT INTO t_message(name, email, msg) VALUES (?, ?, ?)
    <sql:param value="${param.name}"/>
    <sql:param value="${param.email}"/>
    <sql:param value="${param.msg}"/>
  </sql:update>
</c:if>
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>訪客留言版</title>
  </head>
  <body>
    <table style="text-align: left; width: 100%;" border="0"
          cellpadding="2" cellspacing="2">
      <tbody>
        <sql:query sql="SELECT name, email, msg FROM t_message"
                  var="messages"/>
        <c:forEach var="message" items="${messages.rows}">
          <tr>
            <td>${message.name}</td>
            <td>${message.email}</td>
            <td>${message.msg}</td>
          </tr>
        </c:forEach>
      </tbody>
    </table>
  </body>
</html>
```

綜合練習 / 微網誌

- 改用資料庫搭配JDBC存取資料，不過將檔案儲存改為資料庫儲存，就目前應用程式來說，是個不小的變動，因此在這邊將導入DAO（Data Access Object）設計模式，以隔離儲存邏輯與商務邏輯